

Class 06 Assignment: Surfaces

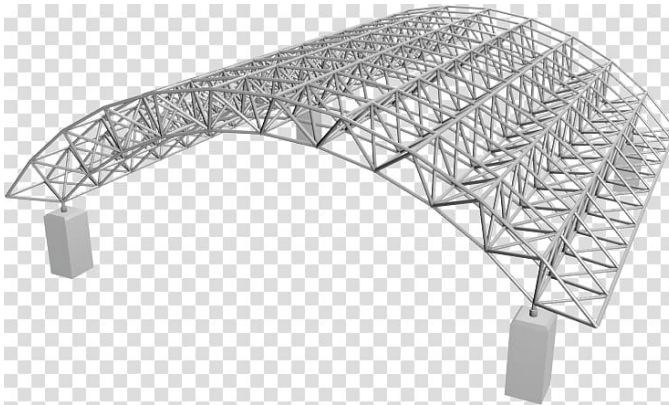
1. Space frame script

Develop a script that given an arbitrary untrimmed surface produces a space frame model. The script should minimally include these inputs:

- s - Surface to serve as the driving surface geometry for the space frame
- a - Number of divisions in u (the number of points in u will be $a+1$)
- b - Number of divisions in v (the number of points in v will be $b+1$)
- d - Offset distance between top and bottom rows of points.

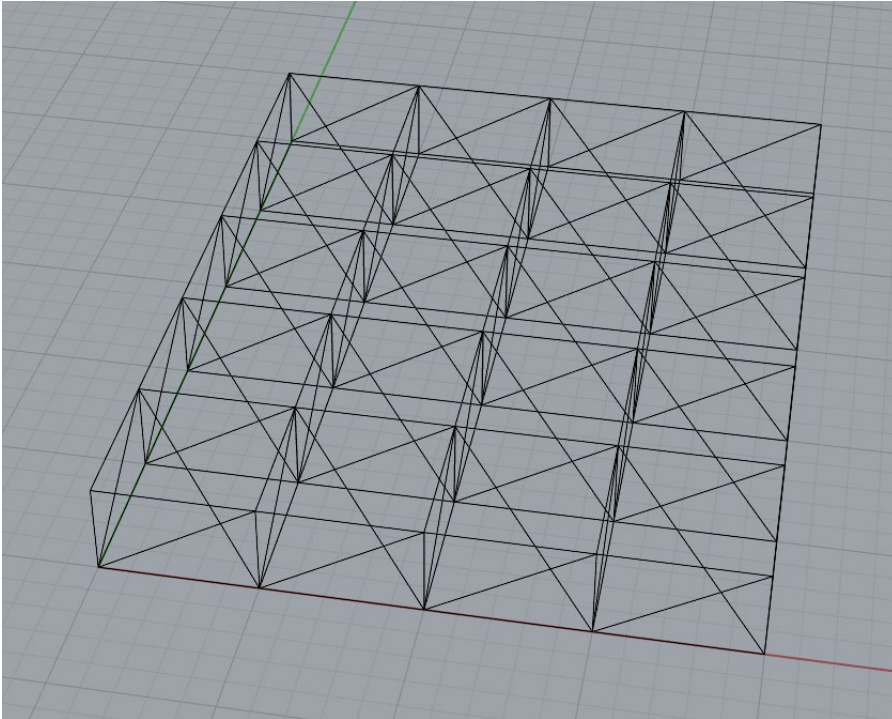
The script should as a general description:

- Produce a $[2] \times [a+1] \times [b+1]$ array of points pts , where
 - the $[a+1] \times [b+1]$ array of points on the surface s is stored in $pts[0][a+1][b+1]$
 - Offset each these points d along the normal vector to the surface at each point in $pts[0]$. These points should be stored in a second layer of the array $pts[1][a+1][b+1]$.
- Generate bar center line lines that appropriately connect between points in pts .
- Once you have successfully completed the space frame wireframe geometry (points and lines) finish your space frame with additional geometry: cylindrical bars centered on each of the wires, spherical nodes at the intersection points, etc.
- Note that there are several types of space frames, that will have different strategies for connecting the nodes in u and v . Here are a few examples:

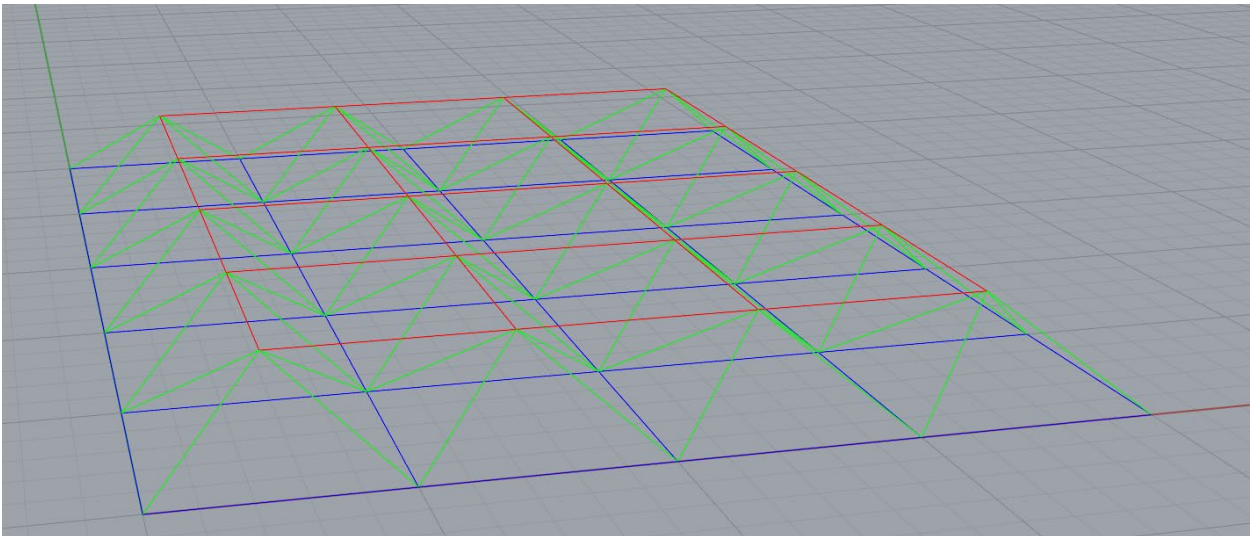


As a general statement, there are two broad types of space frame configurations:

- Space frames where the top and bottom frames are copies of each other, and other bracing connects these two rectangular frames together:



- Perhaps more prevalently, where the upper frame is offset from the lower frame by half a module in both u and v. Algorithmically, this can be achieved either by developing a second grid offset by half a module and offsetting this grid normal to the surface, or (recommended) by connecting even numbered points in the lower grid $pts[0][i][j]$ to odd numbered points in the offset grid $pts[1][i+1][j+1]$, etc.



- Once you have gotten the feel of generating patterns of bars, consider developing a “universal bar generating subroutine”, that can be called to generate all the bars of a given pattern. For example

`generateBars( pointArray, start, step, pattern)`

where:

pointArray is the array [level][xpoints][ypoints]

start is a tuple of integers [zstart, xstart, ystart] representing the starting point of the pattern

step is a tuple of integers [zstep, xstep, ystep] representing how the pattern steps through the array.

pattern is a tuple of integers [zpat, xpat, ypat] representing offset between starting and ending nodes of a given bar.

For example 2 above we could imagine generating the space frame with these and additional calls:

`drawBars(pts, (0,0,0),(2,2,2),(0,2,0))` # generate lower x bars, skipping every other point

...

`drawBars(pts, (1,1,1),(2,2,2),(0,0,2))` # generate upper y bars, skipping every other point

...

`drawBars(pts, (0,0,0),(2,2,2),(1,1,1))` # generate one set of diagonal bars

make sure to rigorously check that you are not “walking off” the point array in either the <0 or >= size directions in x, y, or z!

Place screen captures of your resulting space frame and key parts of the code in copies of [this slide](#) in [this presentation](#).

2. Write 1-2 paragraphs describing in general terms the final project you would like to achieve for the semester and a few of the technical topics that you will need to undertake to achieve this project. Place this text on a slide in the [Assignment 06 assignment presentation](#).